

Name :- Mihir Mungara

Airflow Assignment – 3

DAG Code :-

```
=====

from airflow import DAG

from airflow.sdk import Variable

from datetime import datetime, timedelta

from airflow.providers.amazon.aws.transfers.s3_to_sql import S3ToSqlOperator

from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
```

```
# — Airflow Variables —————

S3_BUCKET = Variable.get("s3_bucket_name")

S3_FILE_KEY = Variable.get("s3_file_key")

TARGET_TABLE = Variable.get("target_table")
```

```
# — Default Arguments —————

default_args = {

    "owner": "airflow",

    "depends_on_past": False,

    "email_on_failure": False,

    "retries": 2,

    "retry_delay": timedelta(minutes=5),

}
```

```
# — DAG Definition —————

with DAG(

    dag_id="blob_storage_to_database",

    default_args=default_args,

    description="Load CSV file from AWS S3 into PostgreSQL",

    schedule="10 12 * * *",

    start_date=datetime(2024, 1, 1),

    catchup=False,
```

```
tags=["s3", "postgres", "etl"],  
) as dag:
```

```
# — Task 1: Create Table —————
```

```
create_table = SQLExecuteQueryOperator(  
    task_id="create_table_if_not_exists",  
    conn_id="postgres_db_conn",  
    sql=f"""  
    CREATE TABLE IF NOT EXISTS {TARGET_TABLE} (  
        id SERIAL PRIMARY KEY,  
        order_id TEXT,  
        customer_id TEXT,  
        region TEXT,  
        product TEXT,  
        order_amount NUMERIC,  
        order_date DATE,  
        order_timestamp TIMESTAMP,  
        loaded_at TIMESTAMP DEFAULT NOW()  
    );  
    """,  
)
```

```
# — Task 2: Load S3 CSV Into PostgreSQL —————
```

```
load_s3_to_db = S3ToSqlOperator(  
    task_id="load_s3_file_to_postgres",  
    aws_conn_id="aws_s3_conn",  
    sql_conn_id="postgres_db_conn",  
    s3_bucket=S3_BUCKET,  
    s3_key=S3_FILE_KEY,  
    table=TARGET_TABLE,
```

```
    column_list=[  
        "order_id",  
        "customer_id",  
        "region",  
        "product",  
        "order_amount",
```

```

        "order_date",

        "order_timestamp",

    ],

    commit_every=1000,

    parser=lambda filename: list(
        __import__("csv").reader(open(filename))
    )[1:], # Skip header row
)

```

— Task 3: Verify Load —————

```

verify_load = SQLExecuteQueryOperator(

    task_id="verify_row_count",

    conn_id="postgres_db_conn",

    sql=f"""

SELECT COUNT(*) AS total_rows

FROM {TARGET_TABLE};

""",

)

```

— Task Dependencies —————

```

create_table >> load_s3_to_db >> verify_load

```

=====

Connections :-

☐	Connection ID ▾	Connection Type ▾	Description ▾	Host ▾	Port ▾	⋮
☐	aws_s3_conn	aws				⌵ ✎ 🗑
☐	postgres_db_conn	postgres		postgres	5432	⌵ ✎ 🗑

Variables :-

☐	Key ▲	Value ▾	Description ▾	Is Encrypted ▾	⋮
☐	s3_bucket_name	my-data-bucket-mm		true	✎ 🗑
☐	s3_file_key	data/input/sales_data.csv		true	✎ 🗑
☐	target_table	public.sales_orders		true	✎ 🗑

DAG Run Details :-

Dag

blob_storage_to_database

Dag Run

manual_2026-05-08T09:50:11.616957+00:00

Search Dags

ctrl+k

Trigger

create_table_if_not_exists

load_s3_file_to_postgres

verify_row_count

2026-05-08 15:20:10

Run Type

Manual

Start Date

2026-05-08 15:20:11

End Date

2026-05-08 15:20:19

Duration

00:00:07.619

Triggering User

airflow

Dag Version(s)

v4

Task Instances

Asset Events

Audit Log

Code

Details

+ Filter

3 Task Instances

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration	
verify_row_count		Success	2026-05-08 15:20:17	1	SQLExecuteQueryOperator	00:00:00.872	
load_s3_file_to_postgres		Success	2026-05-08 15:20:14	1	S3ToSqlOperator	00:00:02.592	
create_table_if_not_exists		Success	2026-05-08 15:20:12	1	SQLExecuteQueryOperator	00:00:01.614	

Logs of Task-1 : Creating Table if not Exists

create_table_if_not_exists

Success

Operator

SQLExecuteQueryOperator

Start Date

2026-05-08 15:20:12

End Date

2026-05-08 15:20:13

Duration

00:00:01.614

Dag Version

v4

Logs

Rendered Templates

XCom

Asset Events

Audit Log

Code

Details

All Log Levels

All Sources

Log message source details sources=["/opt/airflow/logs/dag_id=blob_storage_to_database/run_id=manual_2026-05-08T09:50:11.616957+00:00/task_id=create_table_if_not_exists/attempt=1.log"]

0 [2026-05-08 15:20:12] INFO - DAG bundles loaded: dags-folder

1 [2026-05-08 15:20:12] INFO - Filling up the DagBag from /opt/airflow/dags/blob_to_db_dag.py

2 [2026-05-08 15:20:12] INFO - Executing:
CREATE TABLE IF NOT EXISTS public.sales_orders (
id SERIAL PRIMARY KEY,
order_id TEXT,
customer_id TEXT,
region TEXT,
product TEXT,
order_amount NUMERIC,
order_date DATE,
order_timestamp TIMESTAMP,
loaded_at TIMESTAMP DEFAULT NOW()
);

3 [2026-05-08 15:20:13] INFO - Running statement:
CREATE TABLE IF NOT EXISTS public.sales_orders (
id SERIAL PRIMARY KEY,
order_id TEXT,
customer_id TEXT,
region TEXT,
product TEXT,
order_amount NUMERIC,
order_date DATE,
order_timestamp TIMESTAMP,
loaded_at TIMESTAMP DEFAULT NOW()
);
, parameters: None

Logs of Task-2 : Loading S3 file to postgres

load_s3_file_to_postgres

Success

Operator

S3ToSqlOperator

Start Date

2026-05-08 15:20:14

End Date

2026-05-08 15:20:17

Duration

00:00:02.592

Dag Version

v4

Logs

Rendered Templates

XCom

Asset Events

Audit Log

Code

Details

All Log Levels

All Sources

Log message source details sources=["/opt/airflow/logs/dag_id=blob_storage_to_database/run_id=manual_2026-05-08T09:50:11.616957+00:00/task_id=load_s3_file_to_postgres/attempt=1.log"]

0 [2026-05-08 15:20:14] INFO - DAG bundles loaded: dags-folder

1 [2026-05-08 15:20:14] INFO - Filling up the DagBag from /opt/airflow/dags/blob_to_db_dag.py

2 [2026-05-08 15:20:14] INFO - Loading data/input/sales_data.csv to SQL table public.sales_orders...

3 [2026-05-08 15:20:14] INFO - AWS connection (conn_id='aws_s3_conn', conn_type='aws') credentials retrieved from login and password.

4 [2026-05-08 15:20:16] INFO - Loaded 8 rows into public.sales_orders so far

5 [2026-05-08 15:20:17] INFO - Done loading. Loaded a total of 8 rows into public.sales_orders

Logs of Task-3 : Verifying Row Count

verify_row_count

Success

Operator

Start Date

End Date

Duration

Dag Version

SQLExecuteQueryOperator

2026-05-08 15:20:17

2026-05-08 15:20:18

00:00:00.872

v4

Logs

Rendered Templates

XCom

Asset Events

Audit Log

Code

Details

All Log Levels

All Sources

Log message source details sources=["/opt/airflow/logs/dag_id-blob_storage_to_database/run_id-manual__2026-05-08T09:50:11.616957+00:00/task_id-verify_row_count/attempt-1.log"]

0

[2026-05-08 15:20:18]

INFO

- DAG bundles loaded: dags-folder

1

[2026-05-08 15:20:18]

INFO

- Filling up the DagBag from /opt/airflow/dags/blob_to_db_dag.py

2

[2026-05-08 15:20:18]

INFO

- Executing:
SELECT COUNT(*) AS total_rows
FROM public.sales_orders;

3

[2026-05-08 15:20:18]

INFO

- Running statement:
SELECT COUNT(*) AS total_rows
FROM public.sales_orders;
, parameters: None

4

[2026-05-08 15:20:18]

INFO

- Rows affected: 1

5

[2026-05-08 15:20:18]

INFO

- Pushing xcom ti=RuntimeTaskInstance(id=UUID('019e06fe-a2b7-73c5-ae2d-7418e9221ec0'), task_id='verify_row_count', dag_id='blob_storage_to_database', run_id='manual__2026-05-08T09:50:11.616957+00:00', try_num=

Detailed DAG Run Details :-

manual__2026-05-08T09:50:11.616957+00:00

Success

Logical Date

Run Type

Start Date

End Date

Duration

Triggering User

Dag Version(s)

2026-05-08 15:20:10

Manual

2026-05-08 15:20:11

2026-05-08 15:20:19

00:00:07.619

airflow

v4

Task Instances

Asset Events

Audit Log

Code

Details

State

Success

Run ID

manual__2026-05-08T09:50:11.616957+00:00

Run Type

Manual

Duration

00:00:07.619

Last Scheduling Decision

2026-05-08 15:20:19

Queued At

2026-05-08 15:20:11

Start Date

2026-05-08 15:20:11

End Date

2026-05-08 15:20:19

Data Interval Start

2026-05-08 15:20:10

Data Interval End

2026-05-08 15:20:10

Triggered By

ui

Triggering User

airflow

Version ID

019e06fe-a218-7c0c-8f23-8be9a87620c8

Dag Version(s)

Bundle Name

Created At

dags-folder

2026-05-08 15:20:11

Conf

S3 Bucket Configuration :-

Amazon S3

Buckets

my-data-bucket-mm

data/

input/

input/

Copy S3 URI

Objects

Properties

Objects (1)

Copy S3 URI

Copy S3 URI

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

Name

Type

Last modified

Size

Storage class

sales_data.csv

csv

May 8, 2026, 15:04:38 (UTC+05:30)

562.0 B

Standard

Verifying by Loading Data from Table

```
PS C:\Users\mihir.mungara\Desktop\Airflow_Practice> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5d434364de8e	apache/airflow:3.2.1	"/bin/bash -c 'if [..."	47 minutes ago	Up 27 seconds	8080/tcp	airflow_practice-airflow-init-1
80a9490172ad	postgres:13	"docker-entrypoint.s..."	47 minutes ago	Up 33 seconds (healthy)	5432/tcp	airflow_practice-postgres-1

```
PS C:\Users\mihir.mungara\Desktop\Airflow_Practice> docker exec -it airflow_practice-postgres-1 psql -U airflow -d airflow
psql (13.23 (Debian 13.23-1.pgdg13+1))
Type "help" for help.

airflow=# \dt
airflow=# SELECT * FROM public.sales_orders;
airflow=# SELECT * FROM public.sales_orders;
```

id	order_id	customer_id	region	product	order_amount	order_date	order_timestamp	loaded_at
1	1001	C001	North	Laptop	75000	2023-01-10	2023-01-10 10:15:00	2026-05-08 09:50:16.676772
2	1002	C002	South	Mobile	30000	2023-01-12	2023-01-12 11:20:00	2026-05-08 09:50:16.676772
3	1003	C001	North	Tablet	20000	2023-02-05	2023-02-05 09:10:00	2026-05-08 09:50:16.676772
4	1004	C003	East	Laptop	80000	2023-02-20	2023-02-20 14:45:00	2026-05-08 09:50:16.676772
5	1005	C004	West	Mobile	28000	2023-03-01	2023-03-01 16:30:00	2026-05-08 09:50:16.676772
6	1006	C002	South	Laptop	72000	2023-03-15	2023-03-15 18:25:00	2026-05-08 09:50:16.676772
7	1007	C001	North	Mobile	32000	2023-03-18	2023-03-18 20:10:00	2026-05-08 09:50:16.676772
8	1008	C003	East	Tablet	22000	2023-04-02	2023-04-02 12:00:00	2026-05-08 09:50:16.676772
9	1001	C001	North	Laptop	75000	2023-01-10	2023-01-10 10:15:00	2026-05-08 09:51:14.595675
10	1002	C002	South	Mobile	30000	2023-01-12	2023-01-12 11:20:00	2026-05-08 09:51:14.595675
11	1003	C001	North	Tablet	20000	2023-02-05	2023-02-05 09:10:00	2026-05-08 09:51:14.595675
12	1004	C003	East	Laptop	80000	2023-02-20	2023-02-20 14:45:00	2026-05-08 09:51:14.595675
13	1005	C004	West	Mobile	28000	2023-03-01	2023-03-01 16:30:00	2026-05-08 09:51:14.595675
14	1006	C002	South	Laptop	72000	2023-03-15	2023-03-15 18:25:00	2026-05-08 09:51:14.595675
15	1007	C001	North	Mobile	32000	2023-03-18	2023-03-18 20:10:00	2026-05-08 09:51:14.595675
16	1008	C003	East	Tablet	22000	2023-04-02	2023-04-02 12:00:00	2026-05-08 09:51:14.595675
17	1001	C001	North	Laptop	75000	2023-01-10	2023-01-10 10:15:00	2026-05-08 09:51:14.728749
18	1002	C002	South	Mobile	30000	2023-01-12	2023-01-12 11:20:00	2026-05-08 09:51:14.728749
19	1003	C001	North	Tablet	20000	2023-02-05	2023-02-05 09:10:00	2026-05-08 09:51:14.728749
20	1004	C003	East	Laptop	80000	2023-02-20	2023-02-20 14:45:00	2026-05-08 09:51:14.728749
21	1005	C004	West	Mobile	28000	2023-03-01	2023-03-01 16:30:00	2026-05-08 09:51:14.728749
22	1006	C002	South	Laptop	72000	2023-03-15	2023-03-15 18:25:00	2026-05-08 09:51:14.728749
23	1007	C001	North	Mobile	32000	2023-03-18	2023-03-18 20:10:00	2026-05-08 09:51:14.728749
24	1008	C003	East	Tablet	22000	2023-04-02	2023-04-02 12:00:00	2026-05-08 09:51:14.728749
25	1001	C001	North	Laptop	75000	2023-01-10	2023-01-10 10:15:00	2026-05-08 09:51:29.58361
26	1002	C002	South	Mobile	30000	2023-01-12	2023-01-12 11:20:00	2026-05-08 09:51:29.58361
27	1003	C001	North	Tablet	20000	2023-02-05	2023-02-05 09:10:00	2026-05-08 09:51:29.58361
28	1004	C003	East	Laptop	80000	2023-02-20	2023-02-20 14:45:00	2026-05-08 09:51:29.58361
29	1005	C004	West	Mobile	28000	2023-03-01	2023-03-01 16:30:00	2026-05-08 09:51:29.58361
30	1006	C002	South	Laptop	72000	2023-03-15	2023-03-15 18:25:00	2026-05-08 09:51:29.58361
31	1007	C001	North	Mobile	32000	2023-03-18	2023-03-18 20:10:00	2026-05-08 09:51:29.58361
32	1008	C003	East	Tablet	22000	2023-04-02	2023-04-02 12:00:00	2026-05-08 09:51:29.58361
33	1001	C001	North	Laptop	75000	2023-01-10	2023-01-10 10:15:00	2026-05-08 09:53:50.506293
34	1002	C002	South	Mobile	30000	2023-01-12	2023-01-12 11:20:00	2026-05-08 09:53:50.506293
35	1003	C001	North	Tablet	20000	2023-02-05	2023-02-05 09:10:00	2026-05-08 09:53:50.506293
36	1004	C003	East	Laptop	80000	2023-02-20	2023-02-20 14:45:00	2026-05-08 09:53:50.506293
37	1005	C004	West	Mobile	28000	2023-03-01	2023-03-01 16:30:00	2026-05-08 09:53:50.506293
38	1006	C002	South	Laptop	72000	2023-03-15	2023-03-15 18:25:00	2026-05-08 09:53:50.506293
39	1007	C001	North	Mobile	32000	2023-03-18	2023-03-18 20:10:00	2026-05-08 09:53:50.506293
40	1008	C003	East	Tablet	22000	2023-04-02	2023-04-02 12:00:00	2026-05-08 09:53:50.506293

(40 rows)